A CASE-STUDY BASED REPORT

ON

Credit Card Fraud Detection Using Isolation Forest, SVM, Logistic Regression

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Name: Ada Rohan **Roll No :** 2520030195

Name: Balusupati Praneeth **Roll No :** 2520030186

Name: Gangisetti Darshini **Roll No :** 2520030167

Under the Esteemed guidance of

Dr. Gaddam Ravindra Babu

Assistant Professor, CSE



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract

Introduction .....

Background .....
Motivation for the Project .....
Overview of the Work Carried Out.....
Scope and Responsible-Use Statement.....

Problem Statement .....

Case-Study Scenario.....
Central Case-Study Question .....
Formal Problem Definition .....
Objectives that Guide the Study.....
Assumptions and Limitations .....

Dataset and Exploratory Data Analysis

About the Dataset .....
Purpose of EDA.....
Preparation Steps Actually Performed.....
EDA Exploration - Implementation.....
Summary of EDA Findings.....

Algorithm Selection and Implementation .....

Conclusion

Abstract:

Credit card fraud is a major challenge in digital payment systems because fraudulent transactions can cause financial losses to customers and financial institutions. Machine learning techniques can help identify suspicious transactions by learning patterns from transaction data and detecting unusual activities.

This case study presents a credit card fraud detection system developed using three machine learning algorithms: Logistic Regression, Support Vector Machine (SVM), and Isolation Forest. The application is implemented in Python using Google Colab and supports uploading a dataset in CSV or Excel format. The uploaded data is inspected, preprocessed, and used to train the selected models. The system then generates predictions, evaluates model performance, and displays visualizations to help users understand the results.

The dataset used for the experiment contains 1,000 transaction records and 10 columns, including transaction identifiers, transaction-related features, and the target variable Class. The target variable represents legitimate transactions as 0 and fraudulent transactions as 1. The dataset contains 950 legitimate transactions and 50 fraudulent transactions, demonstrating the class imbalance commonly encountered in fraud detection.

The system evaluates the models using accuracy, precision, recall, F1-score, confusion matrices, and classification reports. It also combines the predictions of the three models using a voting mechanism to categorize transactions into high fraud risk, possible fraud, or legitimate. The experiment illustrates how different machine learning approaches can be combined in a single fraud detection workflow. However, the results obtained from this small sample dataset should not be interpreted as proof of performance on real-world financial transactions.

1. INTRODUCTION

1.1 Background

The widespread adoption of online banking, digital wallets, and electronic payment systems has increased the importance of secure financial transactions. Credit cards provide convenient payment methods, but they may also be misused through unauthorized purchases, stolen card details, and suspicious transaction activities.

Traditional fraud detection systems often depend on predefined rules, transaction limits, and manually specified conditions. Although these methods can identify known suspicious patterns, they may fail to detect unfamiliar patterns or changing fraud behaviour.

Machine learning provides an alternative by using transaction data to identify relationships between transaction characteristics and fraud labels. Supervised learning algorithms can learn from previously labelled transactions, while anomaly detection algorithms can identify transactions that differ from common patterns.

This project implements Logistic Regression, Support Vector Machine (SVM), and Isolation Forest to study their use in credit card fraud detection.

1.2 Motivation for the Project:

The main motivation for this project is to develop a simple and accessible application that demonstrates how machine learning can be applied to financial fraud detection.

The project focuses on the following needs:

- Detecting potentially fraudulent transactions from an uploaded dataset.
- Comparing multiple machine learning techniques within one application.
- Handling missing values and categorical transaction features.
- Evaluating predictions using several classification metrics.
- Displaying fraud detection results through graphs and reports.
- Providing an understandable workflow suitable for academic study.

The project also helps demonstrate the differences between supervised classification and anomaly detection.

1.3 Overview of the Work Carried Out:

The project follows a sequence of data processing, model training, prediction, and evaluation steps.

First, the user uploads a transaction dataset in CSV or Excel format. Python libraries are used to read and inspect the data. The program identifies the target column using commonly used target names, such as Class, Fraud, Target, or Label, and converts the target into a binary representation where 0 indicates a legitimate transaction and 1 indicates fraud.

The data is then prepared for machine learning. Identifier columns are removed, date/time-related fields are processed, categorical features are encoded, missing and infinite values are handled, and numerical features are scaled. The dataset is divided into training and testing sets using an 80:20 stratified split.

Three models are trained: Logistic Regression, SVM, and Isolation Forest. Their predictions are evaluated, and the system generates comparison results, confusion matrices, classification reports, fraud counts, and graphs. Finally, a voting method combines the three model predictions to produce an overall risk category.

1.4 Scope and Responsible-Use Statement:

The scope of this project is limited to demonstrating machine-learning-based credit card fraud detection on uploaded transaction data. It provides a basic experimental workflow rather than a complete production banking security system.

The system is intended for educational and analytical use. Its output should be treated as an indication of potential fraud, not as a definitive financial decision. Real-world deployment would require validation on representative data, monitoring of changing transaction patterns, protection of sensitive financial information, and appropriate human review.

The project does not establish that a transaction is fraudulent solely because an algorithm identifies it as suspicious. False positives may inconvenience legitimate customers, while false negatives may allow fraudulent transactions to remain undetected. Therefore, evaluation and responsible interpretation are important parts of the system.

2. PROBLEM STATEMENT:

2.1 Case-Study Scenario

Consider a financial transaction dataset containing information about transaction amounts, transaction timing, distance from the cardholder's home, recent transaction frequency, merchant risk, international transaction status, card-present status, and device trust.

The dataset contains historical transaction labels indicating whether each transaction is legitimate or fraudulent. The objective is to develop a machine learning workflow that learns from the available data and predicts the class of transactions in a held-out test set.

The case study compares three algorithms with different approaches. Logistic Regression and SVM perform supervised classification using labelled examples. Isolation Forest uses anomaly detection to identify observations that differ from common patterns.

The project presents the results through evaluation metrics and visualizations, enabling a comparison of the methods on the same experimental dataset.

2.2 Central Case-Study Question:

How effectively can Logistic Regression, Support Vector Machine, and Isolation Forest identify fraudulent credit card transactions, and how can their predictions be compared and combined in a single application?

The study also considers whether preprocessing, evaluation metrics, and visualizations can make the fraud detection process easier to understand.

2.3 Formal Problem Definition:

Let the transaction dataset be represented as:

$D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$

where:

- (n) is the number of transaction records.
- (x_i) represents the input features of transaction (i).
- (y_i) represents its target label.
- (y_i = 0) indicates a legitimate transaction.
- (y_i = 1) indicates a fraudulent transaction.

The objective is to develop prediction functions that estimate the class of an unseen transaction using its available features.

For supervised classification, the models learn from the labelled training data. Isolation Forest instead identifies unusual observations, and its anomaly predictions are mapped to the same binary convention for comparison.

The system evaluates predicted labels against the actual test labels. The main measures include accuracy, precision, recall, F1-score, and confusion matrices.

2.4 Objectives that Guide the Study:

The objectives of the project are:

1. To develop a basic credit card fraud detection workflow in Python.
2. To support uploading transaction datasets in CSV and Excel formats.
3. To identify and normalize the target variable into legitimate and fraud classes.
4. To preprocess missing values, categorical variables, and numerical features.
5. To divide the dataset into training and testing subsets using stratified sampling.
6. To train Logistic Regression, SVM, and Isolation Forest models.
7. To evaluate the predictions using accuracy, precision, recall, F1-score, and confusion matrices.
8. To display model outputs through graphs and classification reports.
9. To combine model predictions using a simple voting mechanism.
10. To examine the limitations of the results and identify improvements for future work.

2.5 Assumptions and Limitations:

The project makes several assumptions:

- The uploaded dataset contains transaction records and a target column that identifies legitimate and fraudulent transactions.
- The target labels can be converted into a binary format.
- The available transaction features contain useful information for distinguishing between the two classes.
- The training and testing subsets are created using stratified sampling when the class distribution permits it.
- Isolation Forest anomalies can be mapped to the fraud class for comparative analysis.

The following limitations should also be considered:

- The experimental dataset contains only 1,000 records and may not represent real-world transaction diversity.
- The dataset is imbalanced, with 50 fraud records compared with 950 legitimate records.
- Isolation Forest identifies anomalies rather than learning fraud labels directly.
- The voting mechanism gives each model one vote, regardless of its reliability.
- The implementation does not perform a separate hyperparameter search.
- The implementation does not calculate ROC-AUC as part of its reported evaluation.
- The preprocessing workflow fits the imputer and scaler before the train/test split, which can introduce data leakage into the evaluation.
- Automatically processing columns containing terms such as date or time may require additional validation for numeric time fields.
- Results on a small sample cannot establish the model's effectiveness on future or live transactions.

3. DATASET AND EXPLORATORY DATA ANALYSIS:

3.1 About the Dataset

The project uses a credit card transaction dataset named credit_card_fraud_sample.csv. The dataset contains 1,000 records and 10 columns. It includes transaction-related information and a target variable named Class.

The columns are described below.

Sl. No.	Column Name	Description	Role
1	Transaction_ID	Identifier associated with a transaction	Identifier; removed during preprocessing
2	Time	Transaction time-related field	Input feature, subject to the code's date/time processing
3	Amount	Monetary value of the transaction	Numerical feature
4	Distance_From_Home	Distance between the transaction location and the cardholder's home	Numerical feature
5	Transactions_Last_24H	Number of transactions recorded during the preceding 24 hours	Numerical feature
6	Merchant_Risk_Score	Risk score associated with the merchant	Numerical feature

7	International	Indicates whether the transaction is international	Categorical or binary feature
8	Card_Present	Indicates whether the physical card is present	Categorical or binary feature
9	Device_Trust_Score	Score representing the trust level of the transaction device	Numerical feature
10	Class	Indicates legitimate or fraudulent transaction status	Target variable

The dataset contains 950 legitimate transactions and 50 fraudulent transactions. This represents a class distribution of 95% legitimate transactions and 5% fraudulent transactions.

The source of the sample dataset is not documented in the project materials. Therefore, it should be described as the dataset supplied for the experiment rather than as an officially sourced banking dataset.

3.2 Purpose of EDA:

Exploratory Data Analysis (EDA) is performed to understand the structure, quality, and distribution of the dataset before model training.

The main purposes of EDA are:

- To determine the number of records and columns.
- To identify numerical and categorical features.
- To examine the distribution of the target classes.
- To identify missing values and duplicate records.
- To understand how transaction features vary between legitimate and fraudulent records.
- To identify preprocessing requirements.
- To help interpret the model's predictions and evaluation results.

EDA is important because the quality and distribution of the input data influence the reliability of machine learning models.

3.3 Preparation Steps Actually Performed:

The following preprocessing operations are implemented in the supplied code.

1. **Dataset upload and loading**

The user uploads a CSV or Excel file. The program reads the file using pandas and inspects the dataset's shape, columns, and data types.

2. **Target-column identification**

The program searches for commonly used target-column names, including Class, Fraud, Target, and Label. The selected target is normalized into a binary representation where 0 indicates legitimate transactions and 1 indicates fraud.

3**. Identifier removal**

The code removes identifier columns such as Transaction_ID, ID, and columns ending in _id. These fields are generally not intended to provide meaningful predictive information.

4. **Date/time processing**

Columns containing terms such as date or time are processed to derive components such as year, month, day, and hour. This processing should be checked carefully for datasets in which a column named Time contains numeric elapsed-time values rather than calendar timestamps.

5. **Categorical encoding**

Categorical features are converted into a numerical representation using one-hot encoding so that the algorithms can process them.

6. **Handling invalid and missing values**

Infinite values are replaced with missing values. Missing numerical values are imputed using the median, which is less sensitive to extreme values than the mean.

7. **Feature scaling**

The StandardScaler transformation is applied to numerical input features so that their values are represented on a standardized scale.

8. **Train/test division**

The dataset is divided into 80% training data and 20% testing data. Stratified sampling is used to preserve the approximate class distribution in both subsets. The random state is set to 42 for reproducibility.

An important methodological limitation is that the supplied code fits the imputer and scaler before creating the train/test split. A more reliable experimental design would split the data first and fit preprocessing transformations using only the training data.

3.4 EDA Exploration - Implementation

The supplied project code uses pandas and NumPy for data inspection and preparation, along with Matplotlib and Seaborn for visualizations.

The principal EDA activities include:

- Checking the number of records and columns.
- Inspecting column names and data types.
- Examining missing values.
- Checking duplicate records.

- Studying the target class distribution.
- Comparing selected transaction features between fraud and legitimate classes.
- Displaying graphs that summarize model predictions and fraud counts.

The initial inspection of the supplied sample dataset identified five missing values in Amount and five missing values in Merchant_Risk_Score. No duplicate records were reported in the initial inspection.

The code uses median imputation to handle missing numerical values. The preprocessing workflow also removes identifier columns and encodes categorical features before model training.

The code generates graphs showing the distribution of predicted fraud counts and the results of the three models. Confusion matrices and classification reports provide additional detail about prediction errors.

3.5 Summary of EDA Findings:

The initial analysis of the sample dataset produced the following findings.

Analysis Item	**Finding**
Total records	1,000
Total original columns	10
Legitimate transactions	950
Fraudulent transactions	50
Legitimate class proportion	95%
Fraud class proportion	5%
Missing values in Amount	5
Missing values in Merchant_Risk_Score	5
Duplicate records	None reported in the initial inspection
Identifier handling	Transaction_ID is removed
Target variable	Class
Target encoding	0 = legitimate, 1 = fraud

The initial feature comparison also showed differences between the two transaction classes. The mean transaction amount was approximately 44.77 for legitimate transactions and 188.25 for fraudulent transactions. The mean distance from home was approximately 7.94 for legitimate transactions and 56.65 for fraudulent transactions. The mean merchant risk score was approximately 0.244 for legitimate transactions and 0.788 for fraudulent transactions.

These differences suggest that the supplied sample contains features that may help distinguish the two classes. However, differences in group averages do not establish that a feature independently causes fraud or that it will provide the same predictive value in real-world data.

The EDA findings indicate that class imbalance, missing-value handling, feature preparation, and careful model evaluation are important for this project.

4. ALGORITHM SELECTION AND IMPLEMENTATION:

4.1 Overview of Algorithm Selection:

Three algorithms are selected to examine different approaches to credit card fraud detection.

Algorithm	Learning Approach	Purpose in the Project
Logistic Regression	Supervised classification	Predicts the probability or class of a transaction based on learned feature relationships
Support Vector Machine (SVM)	Supervised classification	Learns a decision boundary to distinguish legitimate and fraudulent transactions
Isolation Forest	Unsupervised anomaly detection	Identifies observations that differ from common transaction patterns

Using these three algorithms allows the project to compare two supervised classification techniques with an anomaly detection technique.

4.2 Logistic Regression

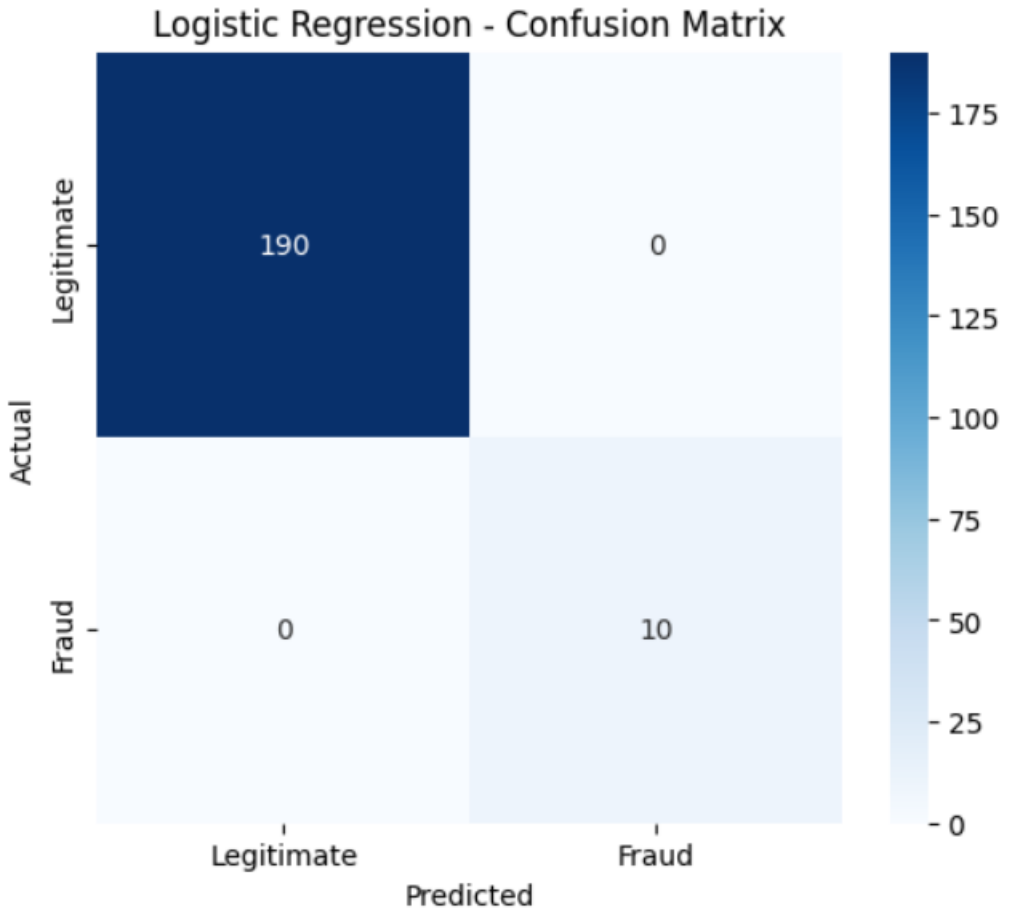
Logistic Regression is a supervised learning algorithm commonly used for binary classification. In this project, it predicts whether a transaction belongs to the legitimate class or the fraud class.

The model estimates the probability of a transaction belonging to the positive class using a logistic function. A decision threshold is then used to convert the estimated probability into a predicted class.

The supplied implementation uses the following configuration:

- Maximum iterations: 1,000.
- Class weighting: balanced.
- Random state: 42.

Balanced class weighting is intended to account for the unequal numbers of legitimate and fraudulent examples in the training data.



Advantages:

- Relatively simple to understand and implement.
- Provides a useful baseline for binary classification.
- Can be efficient on appropriately prepared datasets.
- Supports class weighting for imbalanced data.

Limitations:

- May not represent complex nonlinear relationships without additional feature transformations.
- Its results depend on feature quality and preprocessing.
- High accuracy does not necessarily indicate strong fraud detection performance.

4.3 Support Vector Machine (SVM):

Support Vector Machine is a supervised learning algorithm that identifies a decision boundary separating different classes. It can be useful when the distinction between classes depends on multiple transaction features.

The supplied code uses SVC from scikit-learn with balanced class weights and a fixed random state of 42. The default SVC kernel is the Radial Basis Function (RBF) kernel because no alternative kernel is specified.

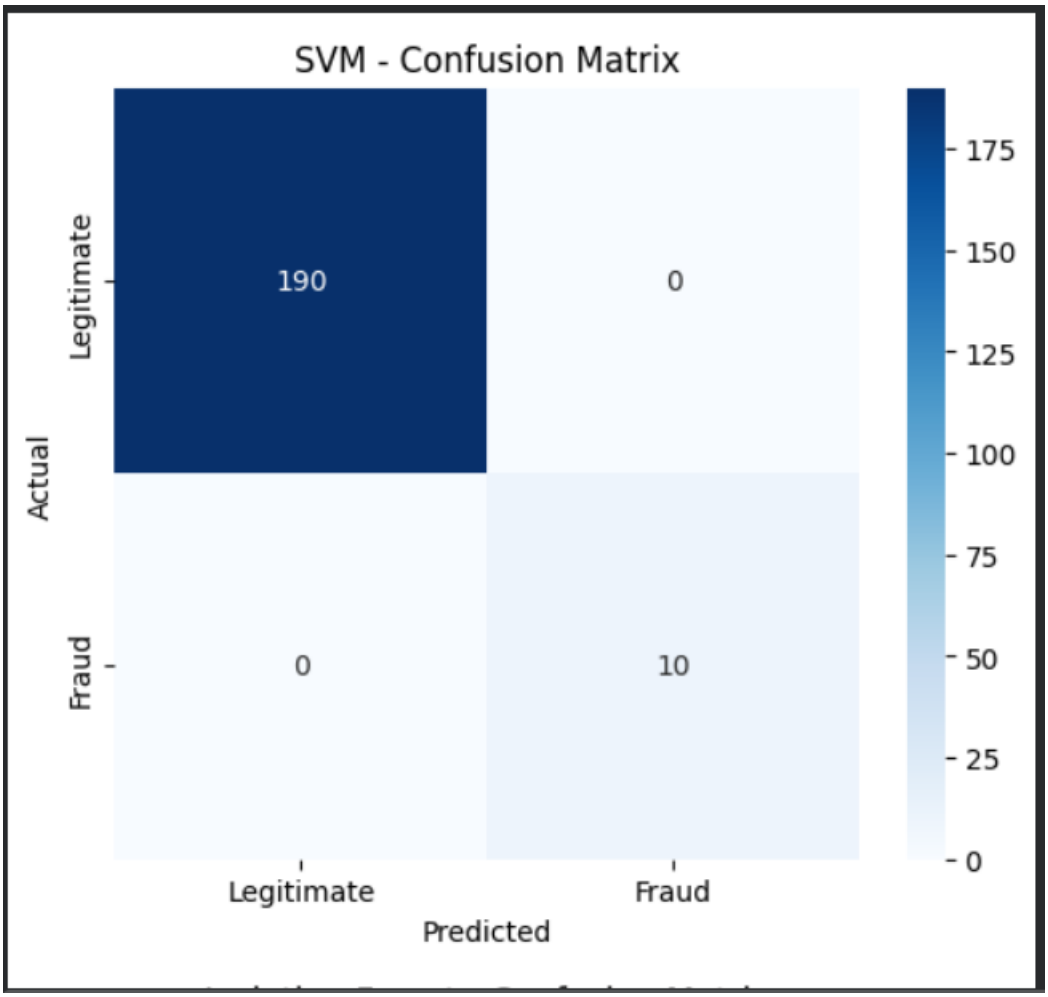
The RBF kernel can represent nonlinear decision boundaries, allowing the model to learn more complex patterns than a simple linear classifier.

Advantages:

- Can model nonlinear relationships using an appropriate kernel.
- Can perform classification using multiple features.
- Supports balanced class weights.

Limitations:

- Training can become computationally expensive for large datasets.
- Performance depends on parameter choices and feature scaling.
- The default settings may not be optimal for every dataset.



4.4 Isolation Forest:

Isolation Forest is an anomaly detection algorithm that identifies observations that are easier to isolate from the rest of the data. Unusual observations may correspond to suspicious transactions, although an anomaly is not automatically fraudulent.

The supplied implementation uses the following settings:

- Number of estimators: 200.
- Random state: 42.
- Contamination: based on the observed fraud proportion in the training labels, clipped to a range between 0.01 and 0.10.

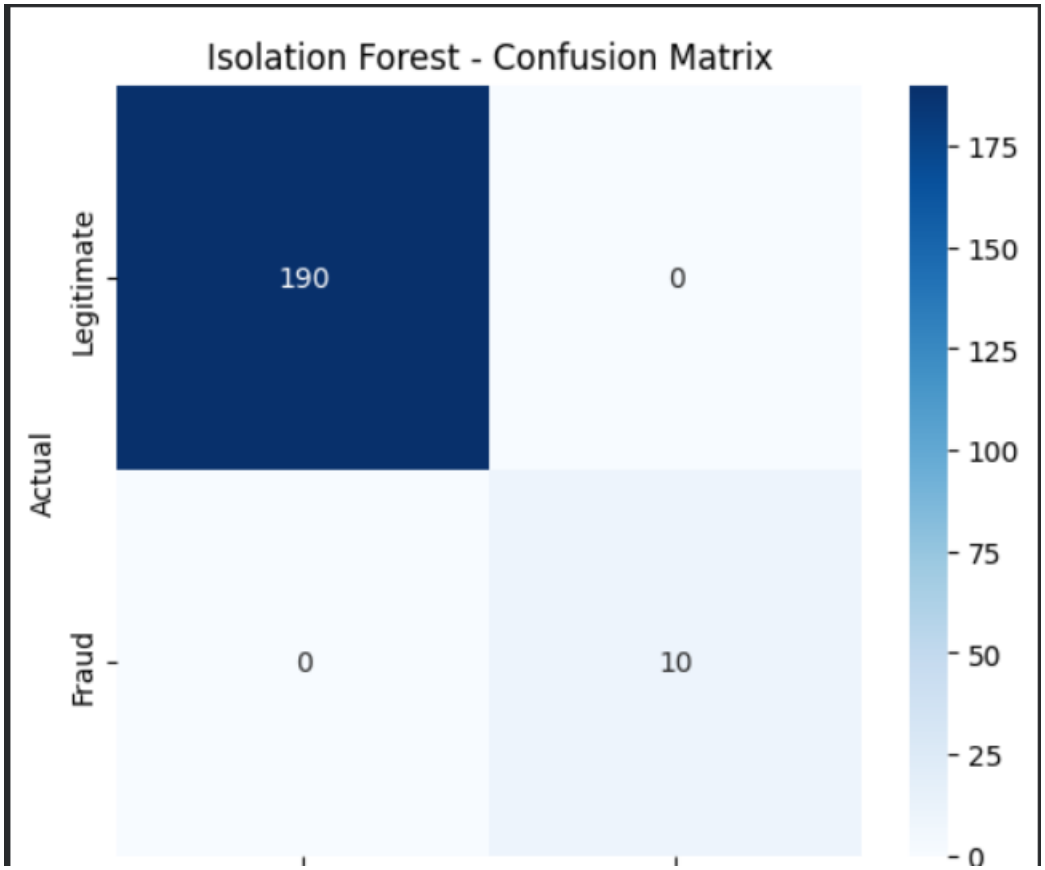
The algorithm produces anomaly labels. The implementation maps the Isolation Forest output of -1 to fraud class 1 and the output of 1 to legitimate class 0.

Advantages:

- Designed for anomaly detection.
- Can identify unusual records without using fraud labels to fit its decision boundary.
- Provides a different perspective from supervised classification models.

Limitations:

- Unusual transactions are not always fraudulent.
- Its output depends on the assumed contamination level.
- Mapping anomaly predictions to fraud labels is a simplifying evaluation choice.
- Its performance should be assessed using representative data and suitable metrics.



4.5 Experimental Setup:

The implementation uses Python and common machine learning and data analysis libraries.

Component	**Implementation**
Programming language	Python
Execution environment	Google Colab
Data processing	pandas and NumPy
Machine learning	scikit-learn
Visualization	Matplotlib and Seaborn
Supported input formats	CSV and Excel
Training/testing ratio	80:20
Sampling method	Stratified train/test split
Random state	42
Target convention	0 = legitimate, 1 = fraud

Evaluation outputs	Accuracy, precision, recall, F1-score, confusion matrix, classification report
Additional outputs	Fraud count, prediction table, three-model voting, graphs

4.6 Model Training and Prediction:

After preprocessing, the program separates the input features from the target labels. It creates training and testing subsets and trains the selected models.

Logistic Regression and SVM learn from the labelled training records. Isolation Forest learns patterns of normality and abnormality from the training features, with its contamination setting derived from the observed training-label fraud proportion.

Each model then generates predictions for the test data. The predicted classes are compared with the actual test labels to calculate evaluation metrics.

The program also presents fraud counts and prediction summaries. These outputs help the user examine how many transactions each model identifies as fraudulent.

4.7 Evaluation Metrics

The project uses the following metrics to assess model performance.

1.Accuracy

Accuracy measures the proportion of all predictions that are correct.

Accuracy = (True Positives + True Negatives) / Total Predictions

2. Precision

Precision measures the proportion of transactions predicted as fraud that are actually fraudulent.

Precision = True Positives / (True Positives + False Positives)

3. Recall

Recall measures the proportion of actual fraudulent transactions correctly identified by the model.

Recall = True Positives / (True Positives + False Negatives)

4. F1-score

The F1-score is the harmonic mean of precision and recall.

F1-score = 2 × (Precision × Recall) / (Precision + Recall)

5. Confusion matrix

The confusion matrix summarizes the number of true positives, true negatives, false positives, and false negatives.

In fraud detection, false positives represent legitimate transactions incorrectly flagged as fraud. False negatives represent fraudulent transactions incorrectly classified as legitimate. Both error types are important when interpreting model results.

4.8 Three-Model Voting Mechanism:

The implementation combines the three model predictions using a simple majority-style voting rule.

Number of Models Predicting Fraud	Final Risk Category
2 or 3	HIGH FRAUD RISK
1	POSSIBLE FRAUD
0	LEGITIMATE

This voting mechanism is intended to provide an easily understandable combined output. It does not train a separate ensemble model, and it assumes that each model contributes one equally weighted vote.

The combined output should therefore be treated as a preliminary risk indicator rather than a calibrated probability of fraud.

4.9 Results and Interpretation:

In the previously reported run on the supplied sample dataset, the 20% test subset contained 200 records, including 190 legitimate transactions and 10 fraudulent transactions.

The reported results for all three models were:

Metric	Logistic Regression	SVM	Isolation Forest
Accuracy	1.0000	1.0000	1.0000
Precision	1.0000	1.0000	1.0000
Recall	1.0000	1.0000	1.0000
F1-score	1.0000	1.0000	1.0000
True negatives	190	190	190
False positives	0	0	0
False negatives	0	0	0
True positives	10	10	10

These are the previously reported experimental results and should be checked against a fresh execution of the supplied notebook before final submission.

Although the reported metrics are perfect, they should not be interpreted as evidence that the algorithms will achieve perfect performance on new or real-world transactions. The dataset is small, the classes may be unusually easy to separate, and the preprocessing workflow fits transformations before the train/test split. These factors limit the strength of the evaluation.

A more reliable assessment would use a leakage-free preprocessing pipeline, independent test data, cross-validation where appropriate, and additional evaluation of false-positive and false-negative behaviour.

5. CONCLUSION:

This case study demonstrates a machine-learning workflow for credit card fraud detection using Logistic Regression, Support Vector Machine, and Isolation Forest. The implementation supports uploading transaction data, identifying the target variable, preprocessing the features, training three models, generating predictions, and presenting evaluation results and visualizations.

The experiment highlights the importance of handling missing values, encoding categorical variables, scaling numerical features, and evaluating both legitimate and fraudulent classes. It also demonstrates how supervised classification and anomaly detection can be compared within a common workflow.

The supplied sample dataset contains 1,000 records, including 950 legitimate transactions and 50 fraudulent transactions. In the previously reported test run, all three models achieved perfect scores on the 200-record test subset. These results are encouraging for demonstrating the implementation, but they require cautious interpretation because the dataset is small and the preprocessing procedure may introduce data leakage.

Overall, the project meets its educational objective of implementing and comparing three machine learning approaches in a single fraud detection workflow. Before the system can be considered for practical financial use, it would need more rigorous validation, improved preprocessing, testing on representative datasets, and additional safeguards for decision-making.

The study demonstrates that a working machine learning application involves more than selecting an algorithm: dataset quality, preprocessing, experimental design, evaluation, and responsible interpretation are equally important.

Signature of the Team members